

labassignment3

February 4, 2025

1 Lab Assignment 3: How to Load, Convert, and Write JSON Files in Python

1.1 DS 6001: Practice and Application of Data Science

1.1.1 Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

1.2 Problem 0

Import the following libraries:

```
[45]: import numpy as np
import pandas as pd
import requests
import json
import sys
sys.tracebacklimit = 0 # turn off the error tracebacks
```

1.3 Problem 1

JSON and CSV are both text-based formats for the storage of data. It's possible to open either one in a plain text editor. Given this similarity, why does a CSV file usually take less memory than a JSON formatted file for the same data? Under what conditions could a JSON file be smaller in memory than a CSV file for the same data? (2 points)

A CSV file usually takes less memory than a JSON formatted file for the same data because CSV files have a simple structure with minimal overhead, consisting of rows of data separated by commas without additional characters for data types or keys.

JSON files include extra characters for keys, braces, brackets, and quotes, which increases the file size. Additionally, JSON files often have redundant key-value pairs, whereas CSV files only store data values.

However, a JSON file could be smaller than a CSV file if the data set is sparse, as JSON only stores non-null fields, or if the data includes complex nested structures that JSON can represent more efficiently.

1.4 Problem 2

NASA has a dataset of all meteorites that have fallen to Earth between the years A.D. 860 and 2013. The data contain the name of each meteorite, along with the coordinates of the place where the meteorite hit, the mass of the meteorite, and the date of the collision. The data is stored as a JSON here: <https://data.nasa.gov/resource/y77d-th95.json>

Look at the data in your web-browser and explain which strategy for loading the JSON into Python makes the most sense and why.

Then write and run the code that will work for loading the data into Python. (2 points)

Given the structure of the JSON data from NASA's meteorite dataset, the most sensible strategy for loading the JSON into Python is to use the `requests` library to fetch the data from the URL and then use the `json` library to parse it.

```
[46]: url = "https://data.nasa.gov/resource/y77d-th95.json"
      response = requests.get(url)

      data = response.json()

      print(data[0])
```

```
{'name': 'Aachen', 'id': '1', 'nametype': 'Valid', 'recclass': 'L5', 'mass':
'21', 'fall': 'Fell', 'year': '1880-01-01T00:00:00.000', 'reclat': '50.775000',
'reclong': '6.083330', 'geolocation': {'type': 'Point', 'coordinates': [6.08333,
50.775]}}
```

1.5 Problem 3

The textbook chapter for this module shows, as an example, how to pull data in JSON format from Reddit's top 25 posts on [/r/popular](https://www.reddit.com/r/popular). The steps outlined there pull all of the features in the data into the dataframe, resulting in a dataframe with 172 columns.

If we only wanted a few features, then looping across elements of the JSON list itself and extracting only the data we want may be a more efficient approach.

Use looping - and not `pd.read_json()` or `pd.json_normalize()` - to create a dataframe with 25 rows (one for each of the top 25 posts), and only columns for `subreddit`, `title`, `ups`, and `created_utc`. The JSON file exists at <http://www.reddit.com/r/popular/top.json>, and don't forget to specify `headers = {'User-agent': 'DS6001'}` within `requests.get()`. (3 points)

```
[47]: url = "http://www.reddit.com/r/popular/top.json"
      headers = {'User-agent': 'DS6001'}
      response = requests.get(url, headers=headers)
      data = response.json()

      # Extract the top 25 posts
      posts = data['data']['children'][:25]

      # Create a list to hold the extracted data
```

```

extracted_data = []

# Loop through the posts and extract the required fields
for post in posts:
    post_data = post['data']
    extracted_data.append({
        'subreddit': post_data['subreddit'],
        'title': post_data['title'],
        'ups': post_data['ups'],
        'created_utc': post_data['created_utc']
    })

# Create a df
df = pd.DataFrame(extracted_data)

print(df.head())
print(f"DataFrame size: {df.shape}")

```

	subreddit	title	ups \
0	pics	A Nazi gets punched in the face.	202178
1	MurderedByWords	How to find Nazis	101 136884
2	pics	Congressmen and protesters outside the USAID	109021
3	MadeMeSmile	He admitted that he had to learn.	102038
4	clevercomebacks	"We'll be fine"	76637

	created_utc
0	1.738604e+09
1	1.738605e+09
2	1.738612e+09
3	1.738623e+09
4	1.738613e+09

DataFrame size: (25, 4)

1.6 Problem 4

The NBA has saved data on all 30 teams' shooting statistics for the 2014-2015 season here: <https://stats.nba.com/js/data/sportvu/2015/shootingTeamData.json>. Take a moment and look at this JSON file in your web browser. The structure of this particular JSON is complicated, but see if you can find the team-by-team data. In this problem our goal is to use `pd.json_normalize()` to get the data into a dataframe. The following questions will guide you towards this goal.

1.6.1 Part a

Download the raw text of the NBA JSON file and register it as JSON formatted data in Python's memory. (2 points)

```

[48]: url = "https://stats.nba.com/js/data/sportvu/2015/shootingTeamData.json"
      response = requests.get(url)

```

```
nba_data = response.json()
```

1.6.2 Part b

Describe, in words, the path that leads to the team-by-team data. (2 points)

1. The root of the JSON object contains a key called **resultSets**.
2. **resultSets** is a list, and the first element of this list is a dictionary.
3. Within this dictionary, there is a key called **rowSet**.
4. **rowSet** contains the actual team-by-team data as a list of lists, where each inner list represents the data for one team.

1.6.3 Part c

Use the `pd.json_normalize()` function to pull the team-by-team data into a dataframe. This is going to be tricky. You will need to use indexing on the JSON data as well as the `record_path` parameter.

If you are successful, you will have a dataframe with 30 rows and 33 columns. The first row will refer to the Golden State Warriors, the second row will refer to the San Antonio Spurs, and the third row will refer to the Cleveland Cavaliers. The columns will only be named 0, 1, 2, ... at this point. (4 points)

```
[49]: # Use pd.json_normalize()
df = pd.json_normalize(nba_data['resultSets'][0], record_path='rowSet')

print(df.head())
print(f"DataFrame size: {df.shape}")
```

	0	1	2	3	4	5	6	7	8	9	\
0	1610612744	Golden State	Warriors	GSW		82	48.7	114.9	14.9	0.498	
1	1610612759	San Antonio	Spurs	SAS		82	48.3	103.5	14.8	0.481	
2	1610612739	Cleveland	Cavaliers	CLE		82	48.7	104.3	16.9	0.481	
3	1610612746	Los Angeles	Clippers	LAC		82	48.6	104.5	15.0	0.497	
4	1610612760	Oklahoma City	Thunder	OKC		82	48.6	110.2	16.1	0.480	

	...	23	24	25	26	27	28	29	30	31	32
0	...	0.478	21.2	42.5	0.497	2.3	6.3	0.363	10.8	25.3	0.429
1	...	0.506	18.3	39.8	0.460	0.9	2.6	0.341	6.1	15.9	0.381
2	...	0.473	18.2	40.7	0.447	1.7	5.7	0.299	9.0	23.9	0.378
3	...	0.480	18.9	42.0	0.450	2.0	6.0	0.334	7.7	20.8	0.373
4	...	0.497	17.5	38.7	0.451	1.6	5.1	0.321	6.6	18.6	0.356

```
[5 rows x 33 columns]
DataFrame size: (30, 33)
```

1.6.4 Part d

Find the path that leads to the headers (the column names), and extract these names as a list. Then set the `.columns` attribute of the dataframe you created in part c equal to this list. The result should be that the dataframe now has the correct column names. (3 points)

```
[50]: # Extract the headers
headers = nba_data['resultSets'][0]['headers']

# Set the columns attribute of the dataframe
df.columns = headers

df.head()
```

```
[50]:
```

	TEAM_ID	TEAM_CITY	TEAM_NAME	TEAM_ABBREVIATION	TEAM_CODE	GP	MIN	\
0	1610612744	Golden State	Warriors	GSW		82	48.7	
1	1610612759	San Antonio	Spurs	SAS		82	48.3	
2	1610612739	Cleveland	Cavaliers	CLE		82	48.7	
3	1610612746	Los Angeles	Clippers	LAC		82	48.6	
4	1610612760	Oklahoma City	Thunder	OKC		82	48.6	

	PTS	PTS_DRIVE	FGP_DRIVE	...	CFGP	UFGM	UFGA	UFGP	CFG3M	CFG3A	\
0	114.9	14.9	0.498	...	0.478	21.2	42.5	0.497	2.3	6.3	
1	103.5	14.8	0.481	...	0.506	18.3	39.8	0.460	0.9	2.6	
2	104.3	16.9	0.481	...	0.473	18.2	40.7	0.447	1.7	5.7	
3	104.5	15.0	0.497	...	0.480	18.9	42.0	0.450	2.0	6.0	
4	110.2	16.1	0.480	...	0.497	17.5	38.7	0.451	1.6	5.1	

	CFG3P	UFG3M	UFG3A	UFG3P
0	0.363	10.8	25.3	0.429
1	0.341	6.1	15.9	0.381
2	0.299	9.0	23.9	0.378
3	0.334	7.7	20.8	0.373
4	0.321	6.6	18.6	0.356


```
[5 rows x 33 columns]
```

1.7 Problem 5

Save the NBA dataframe you extracted in problem 4 as a JSON-formatted text file on your local machine. Format the JSON so that it is organized as dictionary with three lists: `columns` lists the column names, `index` lists the row names, and `data` is a list-of-lists of data points, one list for each row. (Hint: this is possible with one line of code) (2 points)

```
[51]: df.to_json('nba_shooting_stats.json', orient='split')
```